Amardeep Singh

E23CSEU2189

```
# ================================
# LAB 8 – GAN Artistic Image Generation
# ================================

import torch
import torch.nn as nn
import torchvision
import torchvision.transforms as transforms
import torchvision.utils as vutils
import matplotlib.pyplot as plt
import numpy as np

# ================================
# Device configuration
# ================================

device = torch.device("cuda" if torch.cuda.is_available() else "cpu"

# ================================
# Hyperparameters
# ================================

batch_size = 128
image_size = 64
latent_dim = 100
epochs = 5
lr = 0.0002

# ================================
# Data Preparation
# ================================

transform = transforms.Compose([
    transforms.Resize(image_size),
    transforms.ToTensor(),
    transforms.Normalize((0.5,0.5,0.5),(0.5,0.5,0.5))
])

dataset = torchvision.datasets.CIFAR10(
    root='./data',
    train=True
```

```
    train=True,
    download=True,
    transform=transform
)

dataloader = torch.utils.data.DataLoader(
    dataset,
    batch_size=batch_size,
    shuffle=True
)

# ================================
# Generator Model (DCGAN)
# ================================

class Generator(nn.Module):
    def __init__(self):
        super(Generator,self).__init__()

        self.main = nn.Sequential(

            nn.ConvTranspose2d(latent_dim,512,4,1,0,bias=False),
            nn.BatchNorm2d(512),
            nn.ReLU(True),

            nn.ConvTranspose2d(512,256,4,2,1,bias=False),
            nn.BatchNorm2d(256),
            nn.ReLU(True),

            nn.ConvTranspose2d(256,128,4,2,1,bias=False),
            nn.BatchNorm2d(128),
            nn.ReLU(True),

            nn.ConvTranspose2d(128,64,4,2,1,bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(True),

            nn.ConvTranspose2d(64,3,4,2,1,bias=False),
            nn.Tanh()
        )

    def forward(self,x):
        return self.main(x)

# ================================
# Discriminator Model
# ================================

class Discriminator(nn.Module):
```

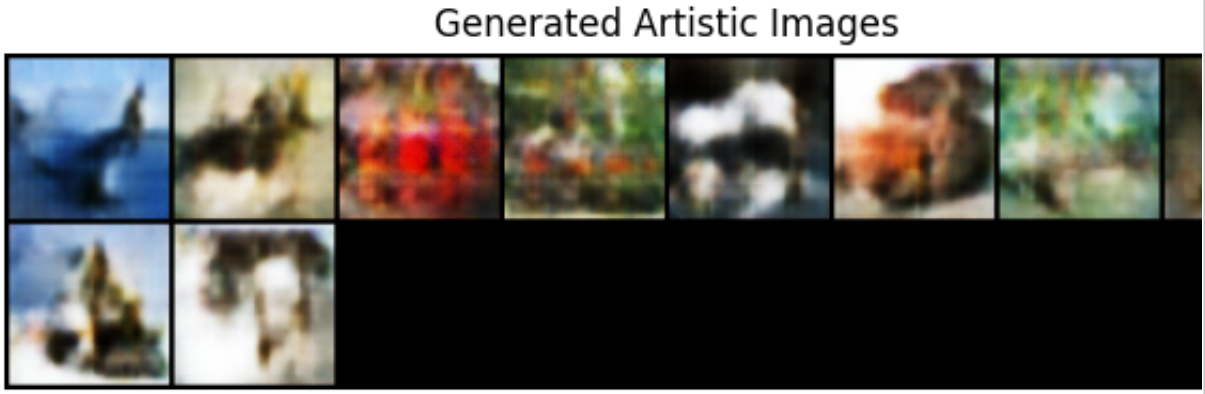
```
    def __init__(self):
        super(Discriminator,self).__init__()

        self.main = nn.Sequential(

            nn.Conv2d(3,64,4,2,1,bias=False),
            nn.LeakyReLU(0.2),

            nn.Conv2d(64,128,4,2,1,bias=False),
            nn.BatchNorm2d(128),
            nn.LeakyReLU(0.2),

            nn.Conv2d(128,256,4,2,1,bias=False),
            nn.BatchNorm2d(256),
            nn.LeakyReLU(0.2),

            nn.Conv2d(256,512,4,2,1,bias=False),
            nn.BatchNorm2d(512),
            nn.LeakyReLU(0.2),

            nn.Conv2d(512,1,4,1,0,bias=False),
            nn.Sigmoid()
        )

    def forward(self,x):
        return self.main(x)

# ================================
# Initialize Models
# ================================

generator = Generator().to(device)
discriminator = Discriminator().to(device)

# ================================
# Loss and Optimizers
# ================================

criterion = nn.BCELoss()

optimizerG = torch.optim.Adam(generator.parameters(), lr=lr, betas=(
optimizerD = torch.optim.Adam(discriminator.parameters(), lr=lr, bet

# ================================
# Training the GAN
# ================================

epochs = 10
```

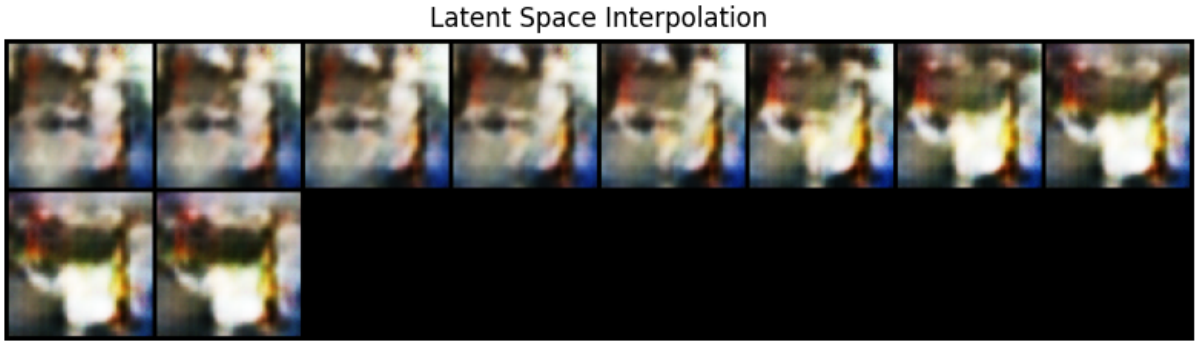
```
print("Training GAN...")

for epoch in range(epochs):

    for i,data in enumerate(dataloader):

        real_images = data[0].to(device)
        batch_size = real_images.size(0)

        real_labels = torch.ones(batch_size,device=device)
        fake_labels = torch.zeros(batch_size,device=device)

        # Train Discriminator

        optimizerD.zero_grad()

        outputs = discriminator(real_images).view(-1)
        loss_real = criterion(outputs,real_labels)

        noise = torch.randn(batch_size,latent_dim,1,1,device=device)
        fake_images = generator(noise)

        outputs = discriminator(fake_images.detach()).view(-1)
        loss_fake = criterion(outputs,fake_labels)

        lossD = loss_real + loss_fake
        lossD.backward()
        optimizerD.step()

        # Train Generator

        optimizerG.zero_grad()

        outputs = discriminator(fake_images).view(-1)
        lossG = criterion(outputs,real_labels)

        lossG.backward()
        optimizerG.step()

    print(f"Epoch [{epoch+1}/{epochs}] LossD: {lossD.item():.4f} Los

# ================================
# Generate Artistic Images
# ================================

print("Generating Artistic Images...")

noise = torch.randn(10,latent_dim,1,1,device=device)
```

```
noise = torch.randn(10,latent_dim,1,1,device=device)
fake_images = generator(noise).detach().cpu()

grid = vutils.make_grid(fake_images, normalize=True)

plt.figure(figsize=(8,8))
plt.axis("off")
plt.title("Generated Artistic Images")
plt.imshow(np.transpose(grid,(1,2,0)))
plt.show()

# ================================
# Latent Space Interpolation
# ================================

print("Latent Space Interpolation...")

z1 = torch.randn(1,latent_dim,1,1,device=device)
z2 = torch.randn(1,latent_dim,1,1,device=device)

alphas = torch.linspace(0,1,10)

images = []

for alpha in alphas:

    z = (1-alpha)*z1 + alpha*z2
    img = generator(z).detach().cpu()
    images.append(img)

images = torch.cat(images)

grid = vutils.make_grid(images, normalize=True)

plt.figure(figsize=(10,5))
plt.axis("off")
plt.title("Latent Space Interpolation")
plt.imshow(np.transpose(grid,(1,2,0)))
plt.show()
```

```
Training GAN...
Epoch [1/10] LossD: 0.3696 LossG: 5.6211
Epoch [2/10] LossD: 5.8036 LossG: 3.6248
Epoch [3/10] LossD: 1.2747 LossG: 6.0487
Epoch [4/10] LossD: 0.7648 LossG: 2.4053
Epoch [5/10] LossD: 0.1038 LossG: 3.6138
Epoch [6/10] LossD: 0.1969 LossG: 3.5306
Epoch [7/10] LossD: 0.0623 LossG: 4.3293
Epoch [8/10] LossD: 0.0259 LossG: 4.6309
Epoch [9/10] LossD: 0.2319 LossG: 5.5607
```

```
Epoch [10/10] LossD: 0.0790 LossG: 4.1930
Generating Artistic Images...
```




```
Latent Space Interpolation...
```